Documentazione della Suite di Test — Albero di Regressione

1. Obiettivi e Ambito del Testing

Il presente documento descrive l'architettura e le schede di dettaglio della suite di test automatizzata realizzata per il progetto **Albero di Regressione** (Java), sviluppata in conformità ai requisiti di qualità del software e alle specifiche architetturali del sistema.

Gli obiettivi primari perseguiti dalla suite sono: - **Correttezza Funzionale e Matematica:** validazione rigorosa degli algoritmi di calcolo della varianza (SSE), della selezione dei migliori punti di split (discreti e continui), dell'ordinamento Quicksort in-place e della generazione delle regole decisionali IF-THEN. - **Integrità dell'Incapsulamento e Polimorfismo:** verifica dell'astrazione ad oggetti (`Attribute`, `Node`, `SplitNode`), dell'information hiding e della serializzazione binaria (`Serializable`, `serialVersionUID`). - **Resilienza e Gestione delle Eccezioni:** verifica esaustiva di tutti i rami di errore previsti (`TrainingDataException`, `UnknownValueException`, `DatabaseConnectionException`, `EmptySetException`), accertando che il sistema non collassi mai e restituisca messaggi chiari all'utente. - **Integrazione Reale ed E2E:** test su database MySQL reale (`MapDB`), socket TCP multithread su porte effimere e chiamate HTTP RESTful complete (`MockRestServiceServer` e Spring Boot Test su porta casuale). - **Assenza Totale di Regressioni:** certificazione del 100% di successo su tutti i 64 test automatizzati (0 fallimenti, 0 errori).

---

2. Tecnologie e Configurazione della Suite

La suite è integrata nel ciclo di vita Maven attraverso il plugin `maven-surefire-plugin` (versione 3.5.2) e fa uso dei seguenti framework: - **JUnit Jupiter 5.11.4:** supporto all'esecuzione dei test, asserzioni avanzate, test annidati (`@Nested`), gestione del ciclo di vita (`@BeforeAll`, `@BeforeEach`, `@AfterEach`, `@AfterAll`) e directory temporanee isolate (`@TempDir`). - **Mockito 5.15.2:** creazione di mock JDBC (`Connection`, `Statement`, `ResultSet`, `ResultSetMetaData`) e intercettazione dei costruttori tramite `Mockito.mockConstruction(DbAccess.class, ...)` per simulare anomalie e schemi non validi senza richiedere permessi speciali sul database. - **Spring Boot Test & Spring WebMVC Test:** esecuzione del container applicativo su porte casuali (`RANDOM_PORT`) per i collaudi End-to-End e test isolati con `MockRestServiceServer` per il client REST.

---

3. Schede dei Test per Modulo e Funzionalità

La suite comprende complessivamente **64 test automatizzati**, suddivisi tra il modulo **Server (44 test)** e il modulo **Client (20 test)**.

MODULO SERVER (44 Test Totali)

3.1 Strutture Dati e Modellazione degli Attributi (`data.AttributeTest` — 6 Test)

Valida la gerarchia polimorfica delle classi del package `data`: - **`testContinuousAttributeInitialization`:** verifica che il costruttore assegni correttamente nome e indice numerico all'attributo continuo e che `toString()` restituisca il nome atteso. - **`testContinuousAttributeSerialization`:** serializza un'istanza di `ContinuousAttribute` in uno stream di byte e la deserializza, verificando l'uguaglianza dei campi di stato. - **`testDiscreteAttributeDeduplicationAndSorting`:** fornisce valori ripetuti e non ordinati (`"Sunny"`, `"Rain"`, `"Sunny"`, `"Overcast"`, `"Rain"`) e accerta che l'iteratore restituisca l'insieme deduplicato e ordinato lessicograficamente (`["Overcast", "Rain", "Sunny"]`), con conteggio dei valori distinti pari a 3. - **`testEmptyAndSingleValueDomains`:** controlla la stabilità della classe con domini discreti vuoti o composti da un solo elemento. - **`testDiscreteAttributeSerialization`:** valida il round-trip di serializzazione e deserializzazione preservando l'integrità del `TreeSet` interno e dell'iteratore `Iterable<String>`. - **`testPolymorphism`:** verifica il corretto dispatch dinamico dei metodi trattando le istanze derivate tramite il tipo astratto `Attribute`.

3.2 Dataset e Ordinamento Quicksort (`data.DataTest` — 12 Test)

Valida l'acquisizione del dataset da MySQL, il partizionamento Quicksort e il rifiuto di dati non conformi: - **`testLoadProvaCSuccessfully`:** effettua una connessione reale al database `MapDB.provaC`, accertando l'istanziazione di 15 tuple con 2 attributi predittori (uno discreto `X` e uno continuo `Y`) e 1 attributo target continuo `C`. - **`testSortDiscreteAttributeFullRange`:** ordina l'intero dataset sull'attributo discreto `X` verificando l'ordinamento in-place (tutte le 'A' prima delle 'B'). - **`testSortContinuousAttributeFullRange`:** ordina il dataset sull'attributo continuo `Y` accertando che ogni elemento risulti minore o uguale al successivo. - **`testSortSubRange`:** ordina un sotto-intervallo arbitrario `[3, 8]` verificando che gli elementi esterni rimangano invariati e quelli interni risultino ordinati. - **`testSortBoundaryRanges`:** controlla la robustezza nei casi limite `begin == end` e `begin > end`, confermando l'assenza di anomalie. - **`testInvalidTableName`:** verifica il rigetto immediato di nomi tabella nulli, vuoti o composti da soli spazi con sollevamento di `TrainingDataException`. - **`testNonExistentTable`:** accerta la corretta gestione dell'errore JDBC in caso di tabella assente nel database. - **`testTableWithLessThanTwoColumns`:** simula tramite mock un dataset con 1 sola colonna, verificando il rifiuto con messaggio esplicito. - **`testLastColumnNotNumeric`:** simula una tabella avente colonna target non numerica (`VARCHAR`), verificando il sollevamento di `TrainingDataException`. - **`testNullValuesInTuple`:** simula la presenza di valori `NULL` all'interno delle tuple, verificando il blocco del caricamento. - **`testNonNumericValueInNumericColumn`:** simula una stringa all'interno di una colonna dichiarata numerica, accertando l'eccezione dedicata. - **`testEmptyTable`:** simula una tabella priva di record, verificando che venga sollevata `TrainingDataException`.

3.3 Persistenza Relazionale e Introspezione Schema (`database.DatabasePackageTest` — 8 Test)

Valida il sottosistema di accesso ai dati relazionali: - **`testColumnProperties`:** controlla costruttore, nome, rappresentazione testuale e predicato `isNumber()` su tipi numerici e testuali. - **`testExampleAddGetIterator`:** valida la struttura dati `Example`, l'inserimento posizionale, la stringa formattata e l'iteratore. - **`testExampleCompareTo`:** verifica la relazione d'ordine totale definita da

`compareTo` su coppie di tuple uguali e distinte. - **`testInitAndCloseConnection`**: apre e chiude una connessione reale a MySQL verificando che risulti attiva dopo `initConnection()` e chiusa dopo `closeConnection()`. - **`testTableSchemaReading`**: interroga i metadati JDBC di `provaC` verificando la correttezza di nomi e tipi delle colonne estratte. - **`testTableDataGetTransazioni`**: estrae le tuple reali dalla tabella restituendole come lista di oggetti `Example`. - **`testGetDistinctColumnValues`**: esegue una query `DISTINCT` ordinata con `TableData.getDistinctColumnValues`, accertando la corrispondenza dei valori univoci (`{"A", "B"}`). - **`testTableDataExceptions`**: verifica che interrogazioni su tabelle inesistenti propaghino correttamente `SQLException`.

3.4 Nodi, Induzione dell'Albero e Calcolo dell'Errore SSE (`tree.TreeNodesAndAlgorithmTest` — 10 Test)

Valida la correttezza matematica dell'albero di regressione: - **`testExactSSEZeroVariance`**: verifica che su un sottoinsieme con varianza nulla (esempi con $X =' B'$, tutti con target $C = 10.0$) il calcolo dia $SSE = 0.0$ con valore predetto 10.0. - **`testExactSSEKnownSubsetA`**: su un sottoinsieme noto con $X =' A'$ calcola la media $\bar{y} = 1.25$ e l'errore $SSE = 0.625$, verificando la perfetta corrispondenza con la formula teorica in `LeafNode`. - **`testLeafNodeProperties`**: accerta che una foglia abbia zero sotto-alberi e che `toString()` inizi con il prefisso `"LEAF : class="`. - **`testDiscreteNodePartitioning`**: crea un nodo di split discreto, verifica la creazione di 2 rami, il calcolo della varianza di split e il corretto routing di `testCondition`. - **`testContinuousNodePartitioning`**: crea un nodo di split continuo su `Y`, accertando la corretta individuazione della soglia di taglio ottima. - **`testContinuousNodeConstantValue`**: controlla il caso limite in cui l'intervallo contiene valori continui identici, generando una singola partizione senza errori. - **`testTreeInductionAndPrints`**: induce l'albero completo su `provaC`, validando l'output di `printTree()` e `printRules()`. - **`testPredictClassSuccessful`**: simula la predizione interattiva tramite stream in memoria, rispondendo alle query e verificando il valore finale stimato (10.0). - **`testPredictClassUnknownValueException`**: invia un'opzione di ramo non valida (`"99"`), verificando il sollevamento controllato di `UnknownValueException`. - **`testSerializationRoundTrip`**: salva l'albero con `salva()` su file temporaneo `.dmp` e lo ricarica con `carica()`, accertando l'identità del modello deserializzato.

3.5 Comunicazione Socket TCP Multithread (`server.SocketClientServerIntegrationTest` — 3 Test)

Valida la componente socket classica end-to-end: - **`testFullFlowAcquireLearnPredict`**: avvia il server su porta TCP effimera e verifica il flusso completo dei comandi `0` (acquisizione), `1` (induzione e salvataggio dump), e `3` (predizione interattiva con esito positivo e valore predetto 10.0). - **`testLoadArchiveAndPredictFlow`**: collauda la persistenza tra client distinti (il primo client addestra e salva il file `.dmp`, il secondo carica il file salvato tramite comando `2` ed effettua la predizione). - **`testErrorHandlingOverSocket`**: verifica che tabelle inesistenti restituiscano la stringa dell'eccezione, che file mancanti segnalino `FileNotFoundException` e che comandi numerici non riconosciuti rispondano con `"Invalid command"`.

3.6 Backend RESTful Spring Boot End-to-End (`server.EndToEndRestAndClientIntegrationTest` — 5 Test)

Valida l'architettura RESTful e la navigazione stateless: - **`testLearnTreeFromDatabaseE2E`:** invia una richiesta HTTP POST a `/api/trees/learn` con payload `{"tableName": "provaC"}`, verificando la risposta `200 OK`, la restituzione delle regole, il popolamento del `rootStep` e la creazione del file `.dmp`. - **`testPredictStepStatelessNavigationE2E`:** esegue una navigazione stateless completa simulando i vari percorsi: - percorso `[1]` (ramo `X=B`): raggiungimento immediato della foglia con valore numerico continuo 10.0; - percorso `[0]` (ramo `X=A`): avanzamento al nodo continuo `Y`; - percorsi `[0, 0]` e `[0, 1]`: raggiungimento delle foglie con stime 1.0 e 1.5. - **`testLoadFromFileAndPredictE2E`:** carica un modello già salvato tramite POST `/api/trees/load` e valida la successiva predizione stateless. - **`testErrorHandlingE2E`:** certifica la conformità delle risposte HTTP di errore: - `400 Bad Request` per tabelle inesistenti o percorsi di navigazione non validi; - `404 Not Found` per archivi `.dmp` non presenti su disco. - **`testOpenApiAndSwaggerUiAvailabilityE2E`:** interroga `/v3/api-docs` validando la specifica JSON OpenAPI 3 con tutti gli endpoint documentati, e verifica che `/swagger-ui.html` risponda con `200 OK`.

MODULO CLIENT (20 Test Totali)

3.7 Acquisizione Dati da Console (`utility.KeyboardTest` — 7 Test)

Valida la classe di utilità per l'input interattivo da terminale: - **`testReadStringAndWord`:** acquisisce righe intere e singole parole da stream bufferizzati in memoria. - **`testReadBoolean`:** valida il parsing insensibile alle maiuscole/minuscole e il fallback a `false` con incremento del contatore errori per input non validi. - **`testReadChar`:** verifica la lettura corretta carattere per carattere. - **`testReadInt`:** effettua il parsing di interi positivi, negativi e zero, con fallback a `Integer.MIN_VALUE` in caso di formato errato. - **`testReadLong`:** valida il parsing a 64 bit e il fallback a `Long.MIN_VALUE`. - **`testReadFloatAndDouble`:** verifica la conversione di numeri con virgola e il fallback a `Float.NaN` e `Double.NaN`. - **`testEndOfLineAndErrorControl`:** controlla la rilevazione di fine riga, il conteggio degli errori e l'abilitazione/disabilitazione dei messaggi di errore.

3.8 Comunicazione HTTP e Client REST (`Client.TreeRestClientServiceTest` — 4 Test)

Collauda in isolamento il client dichiarativo Spring RestClient tramite mock server: - **`testLearnTreeSuccess`:** verifica la corretta composizione della richiesta POST `/api/trees/learn` e la deserializzazione del payload JSON in un record `TreeResponseDto`. - **`testLoadTreeSuccess`:** valida l'invocazione di `/api/trees/load` e l'estrazione delle regole dell'albero caricato. - **`testPredictStepSuccess`:** collauda l'invio del percorso decisionale `path` a `/api/trees/predict` e la conversione in `PredictionStepDto`. - **`testErrorHandlingWithJsonMessage`:** simula una risposta HTTP `400 Bad Request` con corpo `{"error": "Messaggio di errore"}`, accertando che il client estragga il messaggio testuale e sollevi una `RuntimeException` leggibile.

3.9 Visualizzazione Grafica dell'Albero e Regole Mermaid (`Client.TreeMermaidConverterTest` — 5 Test)

Collauda il motore di conversione e parsing visuale delle regole: - **`testParseRulesNominal`**: valida l'estrazione delle regole di `provaC`, scomponendo le condizioni atomiche (`X=A`, `Y<=2.0`) e associandole al valore continuo stimato. - **`testConvertToMermaidNominal`**: verifica la generazione del grafo Mermaid `graph TD` con nodi di split rettangolari, nodi foglia ellittici ed etichette descrittive sui rami. - **`testSingleLeafTree`**: controlla il caso degenere di un albero composto da una sola foglia (nessun bivio), generando un grafo coerente con singolo nodo. - **`testEmptyOrNullInput`**: assicura la resilienza a input nulli o stringhe vuote restituendo un grafo vuoto controllato senza eccezioni. - **`testExtractCleanRules`**: accerta la corretta formattazione della lista di regole leggibili utilizzata per visualizzare le card a schede.

3.10 Apertura Automatica del Browser (`Client.BrowserLauncherTest` — 4 Test)

Valida il componente di avvio automatico del browser all'evento di readiness dell'applicazione: - **`testConfigurationGettersSetters`**: controlla la configurazione della porta server e del flag di auto-apertura del browser. - **`testInvalidUrls`**: verifica la robustezza nei confronti di parametri URL nulli, vuoti o con sintassi non valida. - **`testDisabledAutoOpen`**: verifica che, quando il flag di auto-apertura è disattivato (es. durante l'esecuzione dei test), non venga tentata alcuna apertura del browser. - **`testLaunchViaCommandSimulation`**: collauda i meccanismi di fallback per l'apertura del browser su diversi sistemi operativi (Windows, macOS, Linux).

---

4. Modalità di Esecuzione dei Test

Per eseguire l'intera suite di test automatizzata (64 test):

```
mvn test
```

Oppure tramite lo script rapido nella radice del progetto:

```
test.bat
```

Per eseguire selettivamente i test del server o del client:

```
# Modulo Server (44 test)
mvn -pl Server test

# Modulo Client (20 test)
mvn -pl Client test
```

Per eseguire una singola classe di test specifica (ad esempio `EndToEndRestAndClientIntegrationTest`):

```
mvn -pl Server test -Dtest=EndToEndRestAndClientIntegrationTest
```

---

5. Riepilogo dei Risultati di Esecuzione

L'esecuzione dell'intera suite automatizzata produce un esito positivo al **100% (0 fallimenti, 0 errori)** su tutti i moduli:

- **Modulo Server:** 44 test eseguiti con successo su 6 classi di test (0 fallimenti, 0 errori, tempo medio ~16 s).
- **Modulo Client:** 20 test eseguiti con successo su 4 classi di test (0 fallimenti, 0 errori, tempo medio ~1.5 s).
- **Totale Complessivo: 64 test eseguiti con successo su 10 classi di test (100% PASS, 0 fallimenti, 0 errori)**.